

followed by a read/write bit. The read/write bit indicates whether it wants to read from or write to the slave.

Acknowledge bit (ACK): The addressed slave device responds by pulling the SDA line low during the next clock pulse (SCL). This confirms that the slave is ready to communicate.

Data transmission: The master or slave (depending on the read/write operation) sends data in 8-bit chunks. After each byte, an ACK is sent to confirm that the data has been received successfully.

Stop condition: When the transmission is complete, the master sends a stop condition by releasing the SDA line to HIGH while the SCL line is HIGH. This signals that the communication session has ended.

Repeated Start

Sometimes the master needs to first write something to a device (like a register address), and then immediately read data back from the same device.

If the master sends a STOP and then a new START, another master could take control of the bus in between. That would break the transaction. Instead, the master uses 'Repeated Start'. This is another START condition without sending STOP first. So the bus is not released.

This means that no other master can interrupt and the same device continues the conversation.

Clock stretching: When the slave needs more time

The master controls the clock (SCL) but some devices are slow. Examples are:

- EEPROM writing to flash
- Sensor doing internal conversion

- PMIC updating measurements

If the master keeps sending clocks, the slow device may not be ready.

As discussed earlier, I2C uses open-drain signals so anyone can pull the line LOW and nobody forces it HIGH.

During communication the master releases SCL (tries to make it HIGH). If slave is not ready, it pulls SCL LOW. The clock stays LOW and there is a pause in the communication. When slave is ready, it releases SCL. The pull-up resistor makes SCL HIGH, and the clock continues.

So the slave controls time, not the master.

Arbitration: When two masters talk at the same time

I2C supports multi-master systems. This means both the CPU controller and the BMC can be masters. Both are connected to the same bus.

When both devices start at the same time without arbitration, signals collide, data becomes garbage, and the bus crashes. So, I2C uses bit-level arbitration.

A key rule is that LOW always wins.

Because of open-drain:

- Sending '1' → device releases line (goes HIGH by pull-up)
 - Sending '0' → device pulls line LOW
- If one device sends '1' and another sends '0', the actual bus level equals LOW.

Here's an example of arbitration.

Two masters start at the same time: Master A and Master B both send START (as shown in Figure 5).

Step 1: Both send address bits:

Master A sends bit = 1 (releases SDA)

Master B sends bit = 0 (pulls SDA LOW)

Step 2: Bus becomes LOW. Both read the bus:

Master B: I sent 0 and I see 0 → OK

Master A: I sent 1 but I see 0 → I

lost!

Step 3: Master A stops transmitting. It switches to listening mode.

Step 4: Master B continues alone. No data corruption, no reset and no crash.

This happens bit by bit, not byte by byte. So arbitration is fast and safe.

Limitations of I2C

While I2C is simple, modern systems like smartphones and servers are more demanding. Some of its limitations include:

Limited speed: Even Fast-mode Plus (1MHz) isn't enough for high-throughput sensors or large server systems that need real-time readings.

No in-band interrupts: Devices can't signal the master about urgent events without a separate GPIO interrupt pin.

Fixed addresses: Static addressing can cause conflicts when multiple identical sensors are used — common in large boards like server motherboards.

Higher power use: Pull-up resistors constantly draw current when lines are low, which can add up in energy-sensitive systems.

As embedded system complexity grew, particularly in modern servers with hundreds of sensors and monitoring ICs, these weaknesses drove the development of a more advanced replacement: I3C.

Introducing I3C: Smarter, faster, and more efficient

I3C stands for improved inter-integrated circuit. It keeps the same

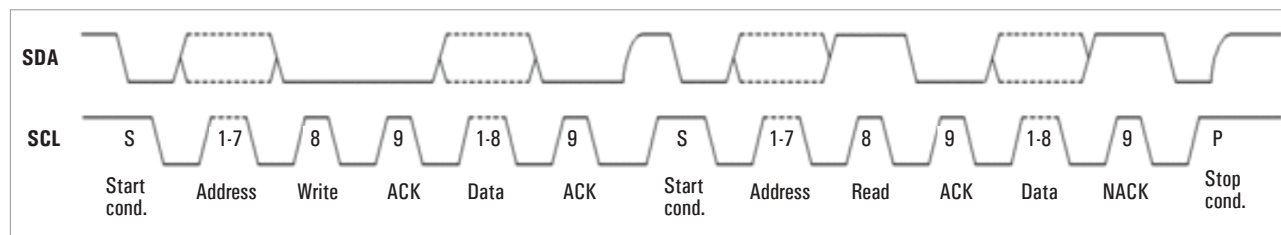


Figure 4: I2C Repeated Start